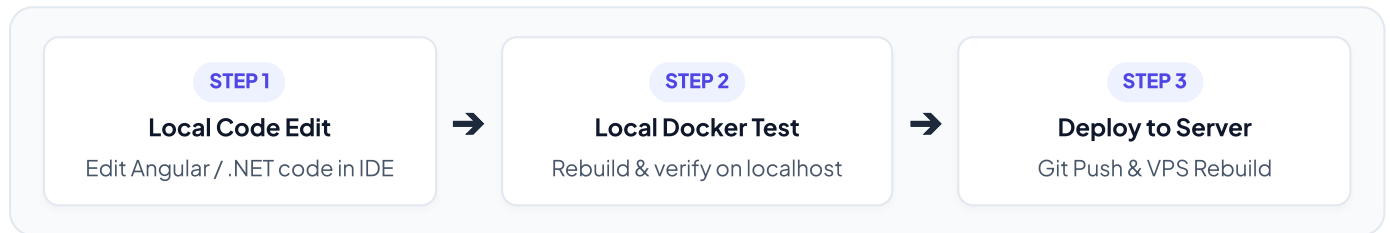


 UPDATE & DEPLOYMENT GUIDE

How to Update Code in Local Docker & Remote Server

Complete step-by-step workflow: how to test changes in local Docker after making code edits, and how to deploy updates to your live production VPS server.

1 The End-to-End Workflow



2 Part 1: How to Update Your Local Docker After Making Edits

Update Local Docker Containers

LOCAL MACHINE

Whenever you modify files in App/ (Frontend) or Server/ (Backend) and want to test them inside your local Docker containers:

1 Rebuild and restart all containers:

```
# Rebuild only the modified code and restart containers
docker-compose up -d --build
```

💡 **Smart Cache:** Docker will only recompile what changed (e.g. if you only edited Angular, it only rebuilds Frontend without touching the Database).

2 Speed Tip — Rebuild only one specific service:

If you only changed the frontend or only the backend, you can rebuild just that single container much faster:

```
# If you only edited Frontend (Angular):
docker-compose up -d --build frontend

# If you only edited Backend (.NET):
docker-compose up -d --build backend
```

3 Verify in your browser:

Open <http://localhost:8080> and do a **Hard Refresh** (Cmd + Shift + R on Mac or Ctrl + F5 on Windows) to clear browser cache.

3 Part 2: How to Deploy & Update the Remote Server (VPS)

Production VPS Deployment (2.24.115.165)

LIVE SERVER

1 On Your Local Machine (Mac): Commit & Push Changes to Git

```
# 1. Stage all your modified files
git add .

# 2. Commit with a message describing what you changed
git commit -m "Update reservation logic and UI fixes"

# 3. Push to GitHub
git push origin main
```

2 Connect to Your VPS Server via SSH:

```
ssh root@2.24.115.165
```

3 On the Server: Pull Changes and Rebuild Containers:

```
# 1. Go into the project directory
cd /root/ReservationSystem

# 2. Pull latest code from GitHub
git pull origin main

# 3. Rebuild and launch the updated containers on Port 80
PORT=80 docker-compose up -d --build
```

 **Zero Downtime on Data:** Your existing database, customers, pilots, and reservations remain 100% safe and intact inside the persistent Docker volume.

4 Verify Live Application:

Open **http://2.24.115.165** in your browser and press **Cmd + Shift + R** to verify your newly deployed changes.

4 Part 3: 1-Click Automated Deployment Script

⚡ Automated Single-Command Update (`. /deploy.sh`)

AUTOMATION

We created a script `deploy.sh` in the project root so you don't even need to manually SSH into the server:

```
# Run this single command from your Mac terminal to push & update server:
./deploy.sh "My latest changes"
```

🌟 This script automatically: (1) Adds and commits your local changes, (2) Pushes to GitHub, (3) SSHs into the VPS, (4) Pulls the updates, and (5) Rebuilds the Docker containers!

5 Quick Reference Command Cheat Sheet

Goal	Where to Run	Exact Command
Update Local Docker	Local Mac	<code>docker-compose up -d --build</code>
Update only Frontend (Local)	Local Mac	<code>docker-compose up -d --build frontend</code>
Update only Backend (Local)	Local Mac	<code>docker-compose up -d --build backend</code>
Push Changes to GitHub	Local Mac	<code>git add . && git commit -m "update" && git push origin main</code>
Update Remote Server (VPS)	VPS Terminal	<code>git pull origin main && PORT=80 docker-compose up -d --build</code>
1-Click Auto Deploy	Local Mac	<code>./deploy.sh "commit message"</code>